

Abstract

Keywords: Static Application Security Testing, Large Language Models,
C programming language, False Positives

Resumen

Keywords: Analizador de código estático, Modelo extenso de lenguaje,
Lenguaje de programación C, Falsos positivos

Índice

1. Introducción	7
2. Estado del arte	9
2.1. El funcionamiento de la pila de llamadas	9
2.2. El funcionamiento del montículo (<i>heap</i>)	9
2.3. Organización de la memoria interna de un proceso	10
2.4. CWEs de la pila de llamadas	11
2.5. CWEs del montículo	12
2.6. CWEs que afectan tanto a la pila de llamadas como al montículo	13
3. Metodología de análisis	15
4. Entrenamiento del gran modelo de lenguaje	17
5. Resultados recopilados	19
6. Conclusiones	21

Índice de figuras

1. Estructura de memoria de un proceso C/C++. Extraída de [6] 10

1

Introducción

El lenguaje de programación C es uno de los lenguajes más populares en la industria de desarrollo de software. Este lenguaje adquiere una notable presencia en sistemas en los que se necesita del acceso a bajo nivel al hardware, la eficiencia temporal y el control explícito de los recursos. Originalmente, C se planteó como un lenguaje de programación de sistemas para la familia de sistemas operativos tipo UNIX, caracterizado técnicamente como un lenguaje liviano, de propósito general y portable [1]. Una gran cantidad de aplicaciones de alto rendimiento y sistemas críticos se implementan en C y continúan en soporte.

Sin embargo, las características que popularizaron a C también son el fondo detrás de una gestión insegura de la memoria intrínseca al lenguaje. Esta gestión insegura propicia la aparición de errores en el manejo de la memoria, que, además de ser difíciles de detectar, constituyen una de las superficies de ataque más persistentes en los lenguajes de programación *memory-unsafe*.

Los lenguajes de programación *memory-unsafe* no cuentan, por defecto, con mecanismos de prevención de errores comunes de programación relacionados con la libertad en el acceso a memoria. Bajo esta libertad, el programador gestiona manualmente la memoria, pero esta libertad conlleva la aparición de distintos tipos de vulnerabilidades, entre las que figuran los desbordamientos de pila, las filtraciones de memoria y la corrupción de memoria.

Concretamente, las vulnerabilidades de corrupción de memoria constituyen el tipo de vulnerabilidad más frecuente. La sobre-escritura de metadatos del montículo, los punteros colgantes, las lecturas de memoria no inicializada, las liberaciones inválidas de memoria o la doble liberación de memoria son debilidades reconocidas por investigadores de seguridad y cuentan con sus respectivas técnicas de protección para abordarlas [2].

Cada una de las vulnerabilidades anteriormente mencionadas se recogen en la Common Weakness Enumeration (CWE), por lo que hacemos énfasis en que las fallas que se encuentran

en programas escritos en C no son errores individuales aislados, sino que son consecuencia del diseño estructural del lenguaje.

A la vista de este escenario, la detección de debilidades supone una necesidad con objeto de asegurar la calidad y diseñar software seguro. La literatura sobre el análisis de programas señala una reducción en el coste de corrección de vulnerabilidades si se detectan tempranamente. De hecho, facilita la implementación de prácticas de seguridad «*shift-left*» e intervenir antes de que los defectos lleguen a la etapa de despliegue o de explotación [3].

Los analizadores de código estático son herramientas que permiten la detección de vulnerabilidades en el código fuente a partir de la detección de patrones. Son herramientas de referencia en la industria de desarrollo software, por ejemplo en las canalizaciones de integración continua, dado que permiten identificar estas vulnerabilidades en las fases tempranas del ciclo de vida del software. Por el contrario, hay estudios empíricos que muestran sus limitaciones en la precisión, cobertura, explicabilidad y unas altas tasas de positivos en sus reportajes [4]. Por lo tanto, detectar estas vulnerabilidades asegurando una alta fiabilidad y utilidad sigue siendo un problema abierto.

Estado del arte

En el contexto de este trabajo profundizaremos en el análisis de vulnerabilidades relacionadas con la pila de llamadas, también llamada pila o *call stack* y el montículo, también llamado *heap*, en el lenguaje C. Ambas son dos regiones de memoria relevantes en la estructura de los programas en tiempo de ejecución. Expondremos el funcionamiento de ambas, y posteriormente explicaremos sus debilidades.

2.1. El funcionamiento de la pila de llamadas

En la pila de llamadas (*stack call*) se almacenan los datos relativos a las subrutinas de un programa. Cuando se invoca una subrutina, se crea en la pila un nuevo marco *stack frame*, que contiene sus parámetros, la dirección de retorno y sus variables locales. Al finalizar la subrutina, su marco se libera.

La pila sigue una estrategia LIFO (*last in, first out*): el último marco en crearse es el primero en destruirse. Por ello, las subrutinas más anidadas (las llamadas más recientes) son las últimas en asignarse pero las primeras en liberarse, de forma análoga a una pila de libros donde solo se puede retirar el libro de la cima.

La gestión de la pila se realiza mediante un puntero de pila. Al insertar un nuevo marco, el puntero se desplaza hasta el tope de dicho marco. Y al salir, el puntero retrocede hasta el tope del marco anterior (o al límite inicial si la pila queda vacía). Estos movimientos son automáticos y el programador no tiene control directo sobre ellos.

2.2. El funcionamiento del montículo (*heap*)

El montículo (*heap*) es una región de memoria dinámica; es decir, los datos almacenados en él se asignan durante el tiempo de ejecución del programa. A diferencia de la pila, los datos del

montículo deben gestionarse manualmente.

La gestión manual de la memoria comprende dos operaciones fundamentales: la petición y la liberación de bloques de memoria. Para solicitar un bloque de un número específico de bytes, se emplean las funciones `malloc()` o `calloc()`, que devuelven la dirección de memoria del bloque asignado. La liberación se realiza mediante la función `free()`, a la que se proporciona como argumento la dirección del bloque que se desea liberar.

Al delegar la gestión de la memoria en el programador, los errores aparecen con una frecuencia mucho mayor. Uno de los más característicos son las **fugas de memoria** (memory leaks), que ocurren cuando no se libera un bloque de memoria tras cumplir su utilidad. En casos severos, puede llevar al agotamiento de los recursos (especialmente en dispositivos con capacidades limitadas), lo que puede derivar en una denegación de servicio [5].

2.3. Organización de la memoria interna de un proceso

Dentro de un proceso, la pila y el montículo comparten la misma región de memoria, con la particularidad de que ambas crecen en direcciones opuestas. Por un lado, la pila tiene un límite de tamaño impuesto por el sistema: en Linux puede crecer hasta 8 MB, mientras que en Windows el límite típico es de 1 MB. En cambio, el montículo puede crecer hasta agotar la memoria virtual disponible. En la figura 1 se observa esta región compartida dentro de la estructura de memoria de un proceso.

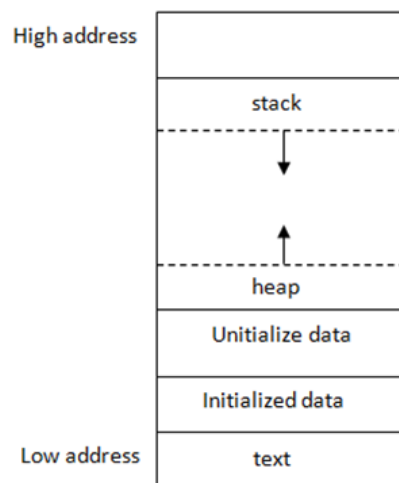


Figura 1: Estructura de memoria de un proceso C/C++. Extraída de [6]

2.4. CWEs de la pila de llamadas

Las debilidades que vamos a tratar se recogen en el catalogo CWE (*Common Weakness Enumeration*) de MITRE, el cual contiene fallos estandarizados conocidos por causar vulnerabilidades. Tanto la gestión de la pila, como la región de memoria en sí, tienen asociadas múltiples debilidades, de forma que en este artículo recogemos las ocho más relevantes para el propósito de nuestro estudio.

- **CWE-121: Desbordamiento del búfer en la pila.** Ocurre cuando se escribe más allá de los límites de un búfer asignado en la pila. Pudiendo corromper datos en marcos adyacentes, como variables o direcciones de retorno, y potencialmente permitir la ejecución de código arbitrario [7].
- **CWE-562: Devolución de puntero de pila.** Ocurre cuando una función devuelve un puntero a una variable local. Al finalizar la ejecución de dicha función, su marco de pila se destruye y todas las referencias a las variables locales que contenía quedan invalidadas. Por tanto, el puntero devuelto apunta a una región de memoria ya desasignada, cuyo uso posterior constituye un comportamiento indefinido y potencialmente explotable [8].
- **CWE-628: Llamada a función con argumentos especificados incorrectamente.** Constituye una categoría base de vulnerabilidades de las cual se desglosan varias vulnerabilidades hijas. Entre las vulnerabilidades hijas podemos encontrar tres específicas de la pila de llamadas.

CWE-683: Llamada a función con argumentos en orden incorrecto.

CWE-685: Llamada a función con número incorrecto de argumentos.

CWE-688: Llamada a función con una variable o referencia incorrecta como argumento.

- **CWE-674: Recursión no controlada.** Se da en el caso de una función recursiva la cual carece de un caso base adecuado o de un límite en su profundidad, lo que lleva a un consumo masivo de memoria de pila (*stack memory*) y al agotamiento de su espacio.
- **CWE-835: Bucle con condición de salida inalcanzable.** Se da cuando el programa entra

en un bucle infinito, lo que lleva a un agotamiento de la pila. Un ejemplo puede ser un bucle `while(true)` sin una instrucción `break`.

2.5. CWEs del montículo

Hay muchos CWEs recogidos que pueden asociarse al montículo, de forma que para nuestro estudio hemos realizado un listado con sus ocho debilidades más relevantes.

- **CWE-122: Desbordamiento del búfer en el montículo.** Ocurre cuando en un bloque de memoria se escribe escriben más datos de los que caben, de forma que compromete la memoria adyacente y puede derivar en la ejecución de código arbitrario.
- **CWE-244: Limpieza incorrecta de la memoria antes de su liberación (*Heap Inspection*).** Ocurre cuando se libera un bloque de memoria (ya sea explícitamente con `free()` o implícitamente con `realloc()`) sin sobrescribir previamente los datos sensibles que contenía. En particular, cuando `realloc()` necesita reubicar el bloque en una nueva dirección copia su contenido, libera el bloque original y lo deja intacto en el *heap*, aunque inaccesible para el programa. Un atacante que pueda leer volcados de memoria o direcciones liberadas y podría recuperar dichos datos [12].
- **CWE-401: Filtraciones de memoria.**
- **CWE-415: Doble liberación.** Se da cuando se llama a la función `free()` dos veces sobre la misma dirección de memoria. Esto corrompe la estructura del montículo y puede derivar en fallos inesperados o comportamiento impredecible; en escenarios controlados por un atacante, puede posibilitar ataques de uso tras liberación (*use-after-free*), que a su vez permiten ejecutar código arbitrario o escalar privilegios [13].
- **CWE-416: Uso tras la liberación (*Use-After-Free*).** Ocurre cuando se accede a una región de memoria tras haber sido liberada, sin que el puntero haya sido reasignado o puesto a `NULL`. Esto provoca que un atacante pueda reutilizar un bloque de memoria liberado, lo que le permite llevar a cabo los ataques previamente descritos [14].
- **CWE-476: Desreferencia de puntero nulo.** Ocurre cuando se emplea un puntero bajo la expectativa de que contenga una referencia a una dirección de memoria válida; no

obstante, el puntero contiene una referencia a NULL, lo que provoca un fallo en el proceso. En circunstancias excepcionales, cuando NULL apunta a la dirección de memoria 0x0 y el código privilegiado puede acceder a ella, de forma que es posible leer o escribir en dicha memoria, puede derivar en la ejecución de código arbitrario [15].

- **CWE-590: Liberación de memoria no asignada en el montículo.** Ocurre cuando se llama a la función `free()` con un puntero de memoria no asignado en el montículo. Al liberar un puntero inválido, pueden corromperse el manejo de las estructuras de datos de memoria, lo que provoca fallos en el programa. En el caso de un ataque, un atacante puede usar la función `free()` para realizar operaciones en memoria de forma que le permitan modificar controles críticos o ejecutar código arbitrario [16].
- **CWE-761: Liberación de puntero no situado al inicio del búfer.** Ocurre cuando se llama a la función `free()` con una dirección de memoria asignada en el montículo, pero que no está al inicio del búfer, lo que provoca un fallo en el programa [17].

2.6. CWEs que afectan tanto a la pila de llamadas como al montículo

Las vulnerabilidades recogidas en este apartado se encuentran asociadas tanto a la pila de llamadas como al montículo, por lo que les hemos dedicado un apartado propio. Las vulnerabilidades asociadas al búfer son muy comunes, por lo que no es de extrañar que tengan mucha prevalencia en el análisis de vulnerabilidades.

- **CWE-123: Escribir lo que sea, donde sea (*Write-what-where*)**
- **CWE-125: Lectura fuera de los límites del búfer.** Ocurre cuando se realiza una lectura de los datos fuera de las direcciones del búfer. De esta debilidad se desglosan otras dos vulnerabilidades hijas:

CWE-126: Lectura de datos más allá del búfer

CWE-127: Lectura de datos atrás del búfer

- **CWE 680: Desbordamiento de entero que desemboca en desbordamiento de búfer**

3

Metodología de análisis

Splint fue diseñado para detectar problemas como:

Desbordamientos de búfer clásicos (CWE-120)

Uso de memoria no inicializada

Pérdidas de memoria

Violaciones de tipos

ASPECTO MUY IMPORTANTE - EL CODIGO NO CUENTA CON ANOTACIONES O
CONTRATOS [POR LO QUE LA EFECTIVIDAD DE SPLINT ES REALMENTE BAJA]

4

Entrenamiento del gran modelo de lenguaje

5

Resultados recopilados

6

Conclusiones

La conclusión

Referencias

- [1] Robert C. Seacord. *Secure coding in C and C++*. eng. 2nd ed. SEI series in software engineering Secure coding in C and C++. Place of publication not identified: Addison Wesley, 2013. ISBN: 9780132981972.
- [2] Vahid Eftekhari Moghadam et al. “Memory Integrity Techniques for Memory-Unsafe Languages: A Survey”. En: *IEEE Access* 12 (2024), págs. 43201-43221. doi: 10.1109/ACCESS.2024.3380478.
- [3] Cristina Cifuentes et al. “The role of program analysis in security vulnerability detection: Then and now”. En: *Computers & Security* 135 (2023), pág. 103463. ISSN: 0167-4048. doi: <https://doi.org/10.1016/j.cose.2023.103463>. URL: <https://www.sciencedirect.com/science/article/pii/S0167404823003735>.
- [4] Bushra Aloraini et al. “An empirical study of security warnings from static application security testing tools”. En: *Journal of Systems and Software* 158 (2019), pág. 110427. ISSN: 0164-1212. doi: <https://doi.org/10.1016/j.jss.2019.110427>. URL: <https://www.sciencedirect.com/science/article/pii/S0164121219302018>.
- [5] Cal Erickson. *Memory leak detection in embedded Systems | Linux Journal*. 2002. URL: <https://www.linuxjournal.com/article/6059>.
- [6] Ruohao Guo Jenny Chen. *CS 225 | Stack and Heap Memory*. URL: <https://courses.grainger.illinois.edu/cs225/sp2020/resources/stack-heap/>.
- [7] *CWE - CWE-121: Stack-based Buffer Overflow (4.20)*. URL: <https://cwe.mitre.org/data/definitions/121.html>.
- [8] *CWE - CWE-562: Return of Stack Variable Address (4.20)*. URL: <https://cwe.mitre.org/data/definitions/562.html>.
- [9] *CWE - CWE-789: Memory Allocation with Excessive Size Value (4.20)*. URL: <https://cwe.mitre.org/data/definitions/789.html>.
- [10] *CWE - CWE-1325: Improperly Controlled Sequential Memory Allocation (4.20)*. URL: <https://cwe.mitre.org/data/definitions/1325.html>.

- [11] *CWE - CWE-193: Off-by-one error (4.20)*. URL: <https://cwe.mitre.org/data/definitions/193.html>.
- [12] *CWE - CWE-244: Improper clearing of heap memory before release ('Heap Inspection') (4.20)*. URL: <https://cwe.mitre.org/data/definitions/244.html>.
- [13] *CWE - CWE-415: Double free (4.20)*. URL: <https://cwe.mitre.org/data/definitions/415.html>.
- [14] *CWE - CWE-416: Use after free (4.20)*. URL: <https://cwe.mitre.org/data/definitions/416.html>.
- [15] *CWE - CWE-476: NULL Pointer Dereference (4.20)*. URL: <https://cwe.mitre.org/data/definitions/476.html>.
- [16] *CWE - CWE-590: NULL Pointer Dereference (4.20)*. URL: <https://cwe.mitre.org/data/definitions/590.html>.
- [17] *CWE - CWE-761: NULL Pointer Dereference (4.20)*. URL: <https://cwe.mitre.org/data/definitions/761.html>.